

EFFICIENCY, DECOMPOSITION, DATA EXAMPLES

COMPUTER SCIENCE MENTORS 61A

October 28 – October 31, 2025

1 Efficiency and Orders of Growth

1. Give a tight asymptotic runtime bound for the following functions in $\Theta(\cdot)$ notation, or “Infinite” if the program does not terminate.

(a)

```
def one(n):  
    while n > 0:  
        n = n // 2
```

(b)

```
def two(n):  
    for i in range(n):  
        for j in range(i):  
            print(str(i), str(j))
```

(c)

```
def three(n):  
    i = 1  
    while i <= n:  
        for j in range(i):  
            print(j)  
        i *= 2
```

2. What do these runtimes simplify to?

$\Theta(8n^2)$	
$\Theta(10000)$	
$\Theta(100n^2 + 10n^3)$	
$\Theta(3n^2 * 2^n)$	
$\Theta(n^{30} + 3^n)$	
$\Theta(n \log n + \log n + n)$	

2 Review

1. Fill in the implementation of `double_link`.

```
def double_link(lnk):
    '''Using mutation, replaces the second in each pair of items
    with the first. The first of each pair stays as is.

    >>> double_link(Link(1, Link(2, Link(3, Link(4)))))
    Link(1, Link(1, Link(3, Link(3))))
    >>> double_link(Link('c', Link('s', Link(6, Link(1, Link('a'))))))
    Link('c', Link('c', Link(6, Link(6, Link('a')))))
    '''

    if _____:
        return _____
    _____
    _____
    return _____
```

2. Write a function that returns `True` if there exists a path from root to leaf that contains at least `n` instances of `elem` in a tree `t`.

*Hint: recall that the built-in function **any** takes in an iterable and returns `True` if any of the iterable's elements are truthy.*

```
def contains_n(elem, n, t):
    '''
    >>> t1 = Tree(1, [Tree(1, [Tree(2)])])
    >>> contains_n(1, 2, t1)
    True
    >>> contains_n(2, 2, t1)
    False
    >>> contains_n(2, 1, t1)
    True
    >>> t2 = Tree(1, [Tree(2), Tree(1, [Tree(1), Tree(2)])])
    >>> contains_n(1, 3, t2)
    True
    >>> contains_n(2, 2, t2) # Not on a path
    False
    '''
    if n == 0:
        return True

    elif _____:
        return _____

    elif _____:
        return _____

    else:
        return _____
```

3. Produce the environment diagram and output that result from executing the code below.

```
def jerry(jerry):  
    def jerome(alex):  
        alex.append(jerry[1:])  
        return alex  
    return jerome
```

```
ben = ['nice', ['ice']]  
jerome = jerry(ben)  
alex = jerome(['cream'])  
ben[1].append(alex)  
ben[1][1][1] = ben  
print(ben)
```

4. Write a generator that, given *m* (the amount of money you have), *pc* (the cost of one pear), and *ac* (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):  
    '''  
    >>> print(list(fruitOptions(10, 2, 5)))  
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear '  
    ',  
    'apple pear pear ', 'apple apple ']  
    '''  
    if _____:  
        yield ''  
    if m >= pc:  
        for _____:  
            _____  
    if m >= ac:  
        for _____:  
            _____
```